

GDB 调试页表翻译流程实验报告

1. 实验目标

使用双重 GDB 方案观察 RISC-V Sv39 页表翻译的完整流程：

- 观察 QEMU 如何实现虚拟地址到物理地址的翻译
- 理解 TLB (Translation Lookaside Buffer) 的工作原理
- 比较 M 态直接映射与 S 态页表翻译的差异

2. 实验环境

2.1 环境配置

- QEMU: qemu-4.1.1 (带调试信息版本)
- GDB:
 - x86_64 gdb (调试 QEMU 进程)
 - riscv64-unknown-elf-gdb (调试 ucore 内核)
- OS: ucore Lab2

2.2 修改 Makefile 使用带调试信息的 QEMU

```
# 使用带调试信息的 QEMU 以支持双重 GDB 调试
QEMU := /root/Downloads/qemu-4.1.1/riscv64-softmmu/qemu-system-riscv64
```

3. QEMU 地址翻译核心源码分析

3.1 关键源码文件

文件	作用
target/riscv/cpu_helper.c	页表翻译核心函数
accel/tcg/cputlb.c	TLB 管理和填充
include/exec/exec-all.h	TLB 数据结构定义

3.2 get_physical_address 函数 (页表遍历核心)

```
// target/riscv/cpu_helper.c:155
static int get_physical_address(CPURISCvState *env, hwaddr *physical,
                               int *prot, target_ulong addr,
                               int access_type, int mmu_idx)
{
    int mode = mmu_idx;
```

```

// M 态不进行页表翻译，直接返回物理地址
if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_MMU)) {
    *physical = addr;
    *prot = PAGE_READ | PAGE_WRITE | PAGE_EXEC;
    return TRANSLATE_SUCCESS;
}

// 获取页表基地址（从 satp 寄存器）
target_ulong base = get_field(env->satp, SATP_PPN) << PGSHIFT;

// 根据 satp.MODE 确定页表级数
vm = get_field(env->satp, SATP_MODE);
switch (vm) {
case VM_1_10_SV39:
    levels = 3; ptidxbits = 9; ptesize = 8; break;
case VM_1_10_SV48:
    levels = 4; ptidxbits = 9; ptesize = 8; break;
// ...
}

// 三级页表遍历 (sv39)
int ptshift = (levels - 1) * ptidxbits; // 初始为 18
for (i = 0; i < levels; i++, ptshift -= ptidxbits) {
    // 计算页表项索引
    target_ulong idx = (addr >> (PGSHIFT + ptshift)) & ((1 << ptidxbits) - 1);

    // 计算页表项物理地址
    target_ulong pte_addr = base + idx * ptesize;

    // 读取页表项
    target_ulong pte = ldq_phys(cs->as, pte_addr);
    target_ulong ppn = pte >> PTE_PPN_SHIFT;

    if (!(pte & PTE_V)) {
        // 无效 PTE
        return TRANSLATE_FAIL;
    } else if (!(pte & (PTE_R | PTE_W | PTE_X))) {
        // 非叶子 PTE，继续遍历下一级
        base = ppn << PGSHIFT;
    } else {
        // 叶子 PTE，翻译完成
        // 设置 A/D 位，返回物理地址
        *physical = (ppn << PGSHIFT) | (addr & ((1 << (PGSHIFT + ptshift)) - 1));
        return TRANSLATE_SUCCESS;
    }
}
return TRANSLATE_FAIL;
}

```

3.3 riscv_cpu_tlb_fill 函数 (TLB 填充)

```
// target/riscv/cpu_helper.c:435
bool riscv_cpu_tlb_fill(CPUState *cs, vaddr address, int size,
                        MMUAccessType access_type, int mmu_idx,
                        bool probe, uintptr_t retaddr)
{
    hwaddr pa = 0;
    int prot;
    int ret;

    // 调用 get_physical_address 进行页表遍历
    ret = get_physical_address(env, &pa, &prot, address, access_type, mmu_idx);

    if (ret == TRANSLATE_SUCCESS) {
        // 翻译成功, 填充 TLB
        tlb_set_page(cs, address & TARGET_PAGE_MASK, pa & TARGET_PAGE_MASK,
                    prot, mmu_idx, TARGET_PAGE_SIZE);
        return true;
    } else {
        // 翻译失败, 触发 Page Fault
        raise_mmu_exception(env, address, access_type, pmp_violation);
        riscv_raise_exception(env, cs->exception_index, retaddr);
    }
}
```

4. 调试结果分析

4.1 调试方法

1. 启动 QEMU (带调试选项)

```
make debug
```

2. 附加 x86_64 GDB 到 QEMU 进程

```
gdb -q
(gdb) attach <QEMU_PID>
(gdb) handle SIGPIPE nostop noprint
(gdb) break get_physical_address
(gdb) continue
```

3. 启动 riscv64 GDB 连接 ucore

```
make gdb
(gdb) continue
```

4.2 M 态访存 (无页表翻译)

在 M 态 (mmu_idx = 3), 虚拟地址直接作为物理地址:

```
=== get_physical_address 被调用 ===  
虚拟地址 addr = 0x80000000  
访问类型 access_type = 2 (INST_FETCH)  
mmu_idx = 3 (M 态)  
  
结果: 直接返回 physical = 0x80000000, prot = 7 (RWX)
```

关键代码路径:

```
if (mode == PRV_M || !riscv_feature(env, RISCV_FEATURE_MMU)) {  
    *physical = addr;  
    *prot = PAGE_READ | PAGE_WRITE | PAGE_EXEC;  
    return TRANSLATE_SUCCESS;  
}
```

4.3 S 态访存 (Sv39 页表翻译)

在 S 态 (mmu_idx = 1), 需要进行三级页表遍历:

```
=== get_physical_address 被调用 ===  
虚拟地址 addr = 0xffffffffc0200000  
访问类型 access_type = 2 (INST_FETCH)  
mmu_idx = 1 (S 态)  
  
调用栈:  
#0 get_physical_address (addr=0xffffffffc0200000, mmu_idx=1)  
   at cpu_helper.c:158  
#1 riscv_cpu_tlb_fill (address=0xffffffffc0200000, access_type=MMU_INST_FETCH)  
   at cpu_helper.c:451  
#2 tlb_fill (addr=0xffffffffc0200000, access_type=MMU_INST_FETCH)  
   at cputlb.c:878
```

Sv39 虚拟地址结构:

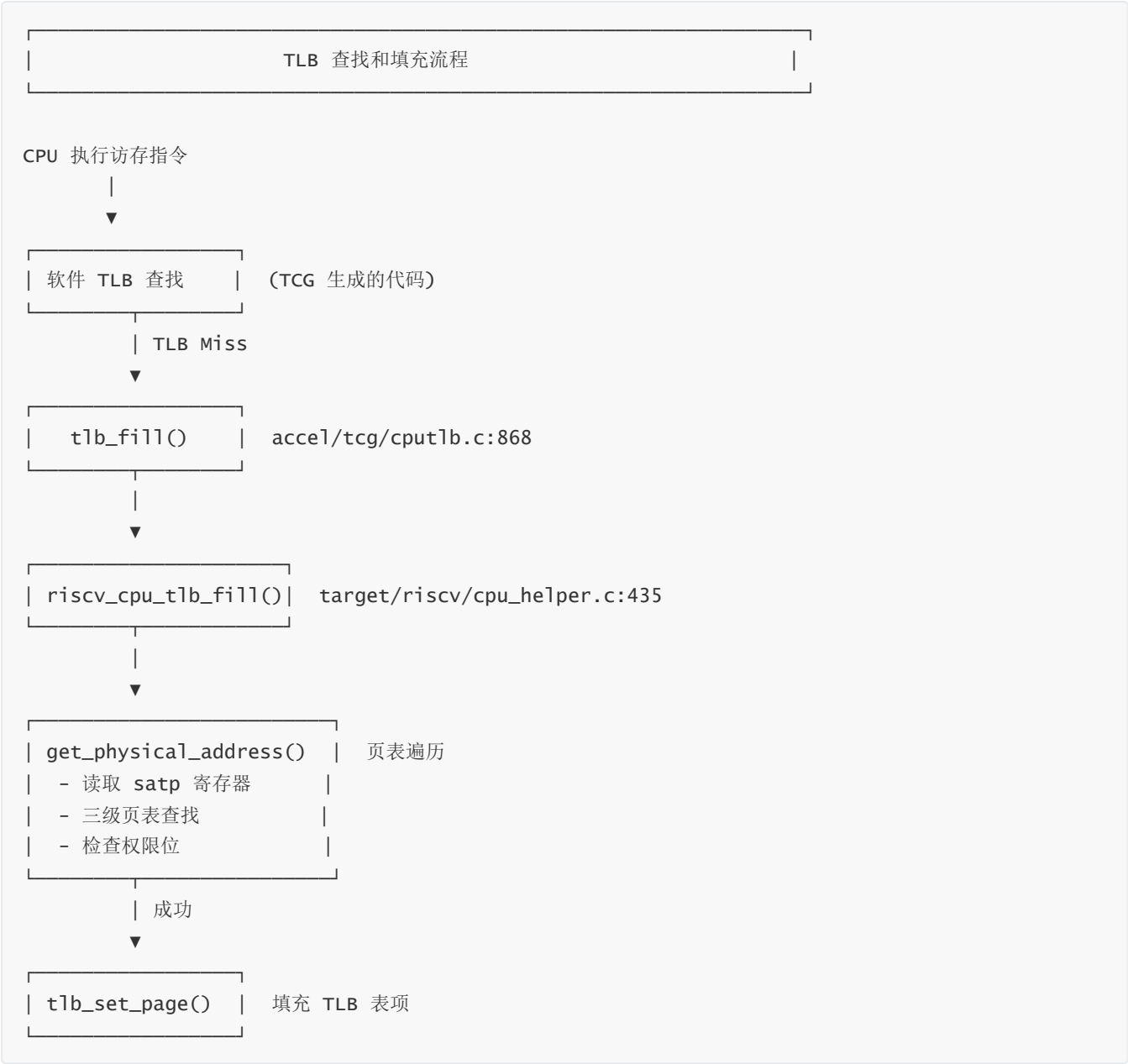
63	39	38	30	29	21	20	12	11	0
+-----+-----+-----+-----+-----+									
sign		VPN[2]			VPN[1]		VPN[0]		offset
Ext		(9bit)			(9bit)		(9bit)		(12bit)
+-----+-----+-----+-----+-----+									

对于 0xffffffffc0200000:

- VPN[2] = 0x100 (256)
- VPN[1] = 0x001 (1)
- VPN[0] = 0x000 (0)
- offset = 0x000

4.4 TLB 填充过程

当 TLB 未命中时，调用链如下：



4.5 观察到的典型地址翻译

从调试输出中观察到的地址翻译示例：

虚拟地址	物理地址	说明
0xffffffffc0200000	0x80200000	内核代码段起始
0xffffffffc0201000	0x80201000	内核代码
0xffffffffc0205000	0x80205000	页表本身 (satp)
0xffffffffc0206000	0x80206000	内核数据段

地址映射规律:

虚拟地址 = 0xffffffff00000000 + 物理地址
即: VA = PA + 0xffffffff40000000 (对于内核空间)

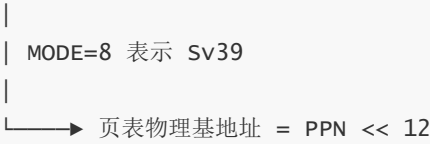
5. Sv39 三级页表翻译详解

5.1 页表结构

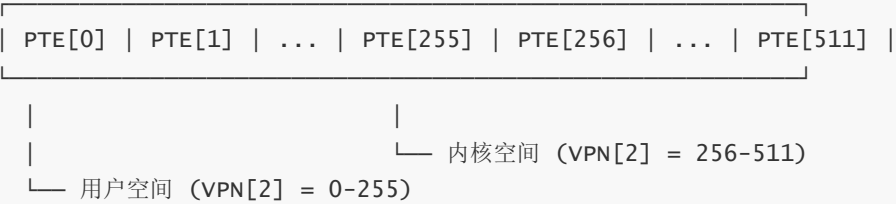


satp 寄存器

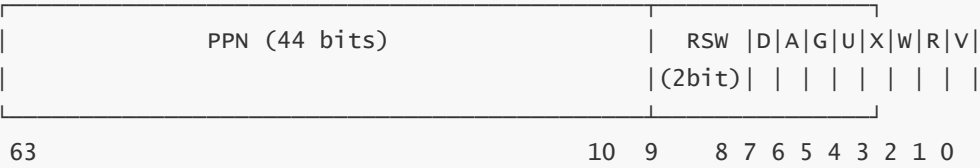
MODE	ASID	PPN
(4)	(16)	(44)



第一级页表 (根页表)



页表项 (PTE) 格式 (64位):



- V = valid (有效位)
- R = Readable (可读)
- W = Writable (可写)
- X = Executable (可执行)
- U = User (用户态可访问)
- G = Global (全局映射)
- A = Accessed (已访问)
- D = Dirty (已修改)

5.2 地址翻译过程

以翻译虚拟地址 `0xffffffffc0200000` 为例:

步骤 1: 解析虚拟地址

```
VA = 0xffffffffc0200000
VPN[2] = (VA >> 30) & 0x1FF = 0x100 (256)
VPN[1] = (VA >> 21) & 0x1FF = 0x001
VPN[0] = (VA >> 12) & 0x1FF = 0x000
Offset = VA & 0xFFF = 0x000
```

步骤 2: 第一级页表查找

```
satp.PPN = 0x80205 (假设)
L1_base = 0x80205000
L1_index = VPN[2] = 256
PTE_addr = L1_base + 256 * 8 = 0x80205800
读取 PTE1, 获取 L2 页表基地址
```

步骤 3: 第二级页表查找

```
L2_base = PTE1.PPN << 12
L2_index = VPN[1] = 1
PTE_addr = L2_base + 1 * 8
读取 PTE2, 获取 L3 页表基地址
```

步骤 4: 第三级页表查找 (或使用大页)

```
L3_base = PTE2.PPN << 12
L3_index = VPN[0] = 0
PTE_addr = L3_base + 0 * 8
读取 PTE3, 这是叶子节点
```

步骤 5: 生成物理地址

```
PA = (PTE3.PPN << 12) | Offset
PA = 0x80200000
```

6. 开启/未开启虚拟地址的访存差异

6.1 M 态 (mmu_idx = 3) - 直接映射

```
// 代码路径: get_physical_address()
if (mode == PRV_M) {
    *physical = addr; // 虚拟地址 = 物理地址
    *prot = PAGE_READ | PAGE_WRITE | PAGE_EXEC;
    return TRANSLATE_SUCCESS;
}
```

特点:

- 无需页表遍历
- 访问任意物理地址
- 无权限检查 (除 PMP)

- 性能最高

6.2 S 态 (mmu_idx = 1) - Sv39 页表翻译

特点:

- 需要三级页表遍历
- 每次 TLB miss 需要 3 次内存访问
- 支持权限控制 (R/W/X)
- 支持用户/内核隔离

6.3 性能对比

访存性能对比

M 态访存:

CPU → 物理内存
延迟: 1 次内存访问

S 态访存 (TLB Hit):

CPU → TLB → 物理内存
延迟: 1 次内存访问 + TLB 查找

S 态访存 (TLB Miss):

CPU → TLB Miss → 页表遍历 (3次内存访问) → TLB 填充 → 物理内存
延迟: 4 次内存访问

注: QEMU 使用软件 TLB, 实际硬件有硬件 TLB, 性能更高

7. QEMU 软件 TLB 实现

7.1 TLB 结构

```
// include/exec/cpu-defs.h
typedef struct CPUTLBEntry {
    target_ulong addr_read;    // 读地址 (虚拟地址)
    target_ulong addr_write;   // 写地址
    target_ulong addr_code;    // 代码执行地址
    uintptr_t addend;          // 物理地址偏移
} CPUTLBEntry;
```


7.2 TLB 查找 (TCG 生成的代码)

```
// 简化的 TLB 查找逻辑
tlb_entry = &env->tlb_table[mmu_idx][TLB_INDEX(addr)];
if (likely(tlb_entry->addr_read == (addr & TARGET_PAGE_MASK))) {
    // TLB Hit
    physical_addr = addr + tlb_entry->addend;
} else {
    // TLB Miss, 调用 tlb_fill()
    tlb_fill(cpu, addr, size, access_type, mmu_idx, retaddr);
}
```

8. 调试命令总结

8.1 启动调试环境

```
# 终端 1: 启动 QEMU
cd /root/OSlab/NKU_OS/labcode\(\latest\)lab2
make debug

# 终端 2: 附加到 QEMU
QEMU_PID=$(pgrep -f qemu-system-riscv64)
gdb -q
(gdb) attach $QEMU_PID
(gdb) handle SIGPIPE nostop noprint
(gdb) break get_physical_address
(gdb) continue

# 终端 3: 连接 ucore
make gdb
(gdb) continue
```

8.2 有用的 GDB 命令

```
# 在地址翻译函数设置断点
break get_physical_address
break riscv_cpu_tlb_fill
break tlb_set_page

# 打印翻译参数
print/x addr
print access_type
print mmu_idx

# 查看调用栈
backtrace

# 条件断点 (只在 S 态停下)
break get_physical_address if mmu_idx == 1
```

9. 总结

通过本次 GDB 调试实验，深入理解了：

1. QEMU 如何实现 Sv39 页表翻译：

- `get_physical_address()` 函数实现三级页表遍历
- M 态直接映射，无需页表
- S/U 态需要完整的页表遍历

2. TLB 的工作原理：

- 软件 TLB 缓存翻译结果
- TLB Miss 时调用 `riscv_cpu_tlb_fill()`
- 翻译成功后通过 `tlb_set_page()` 填充 TLB

3. 虚拟地址与物理地址的映射：

- 内核空间: $VA = PA + 0xffffffff40000000$
- 用户空间: 由页表决定映射关系

4. 权限控制：

- PTE 中的 R/W/X/U 位控制访问权限
- 违反权限触发 Page Fault

这些知识对于理解操作系统的内存管理机制至关重要。